

CSE 216 — Advanced SQL Lab Test Prep (MySQL)

50 practice questions with MySQL solutions • Rounds 1–7 • Compiled for lab test review

Assumed Schema

```
employees(employee_id, first_name, last_name, salary, department_id, job_id, manager_id, hire_date)
departments(department_id, department_name, manager_id)
jobs(job_id, job_title)
salaries(employee_id, salary, salary_date) -- salary history, used in Q28
full_name = CONCAT(first_name, ' ', last_name). MySQL 8.0.31+ used for INTERSECT/EXCEPT/window functions/CTEs.
```

Section A — Subqueries (1–10)

Q1. Employees earning strictly more than the overall average salary. Display EMPLOYEE_ID, FULL_NAME, SALARY, DEPARTMENT_NAME. Order by SALARY desc.

```
SELECT e.employee_id, CONCAT(e.first_name, ' ', e.last_name) AS full_name,
       e.salary, d.department_name
FROM employees e JOIN departments d ON e.department_id = d.department_id
WHERE e.salary > (SELECT AVG(salary) FROM employees)
ORDER BY e.salary DESC;
```

Q2. Employees whose salary equals the company max salary. No LIMIT/FETCH.

```
SELECT e.employee_id, CONCAT(e.first_name, ' ', e.last_name) AS full_name,
       e.salary, d.department_name
FROM employees e JOIN departments d ON e.department_id = d.department_id
WHERE e.salary = (SELECT MAX(salary) FROM employees);
```

Q3. Employees whose salary is below overall average but above overall minimum. Display employee_id, last_name, salary, department_name.

```
SELECT e.employee_id, e.last_name, e.salary, d.department_name
FROM employees e JOIN departments d ON e.department_id = d.department_id
WHERE e.salary < (SELECT AVG(salary) FROM employees)
      AND e.salary > (SELECT MIN(salary) FROM employees);
```

Q4. Second-highest salary in the company. No LIMIT/TOP/FETCH.

```
SELECT employee_id, CONCAT(first_name, ' ', last_name) AS full_name, salary, job_title
FROM employees e JOIN jobs j ON e.job_id = j.job_id
WHERE salary = (
    SELECT MAX(salary) FROM employees
    WHERE salary < (SELECT MAX(salary) FROM employees)
);
```

Q5. Third-highest distinct salary in the company.

```
SELECT employee_id, full_name, salary, department_name FROM (
    SELECT e.employee_id, CONCAT(e.first_name, ' ', e.last_name) AS full_name,
           e.salary, d.department_name,
           DENSE_RANK() OVER (ORDER BY e.salary DESC) AS rnk
    FROM employees e JOIN departments d ON e.department_id = d.department_id
) t
WHERE rnk = 3;
```

Q6. Employees whose salary is greater than every salary in department 5. Use ALL.

```
SELECT employee_id, CONCAT(first_name, ' ', last_name) AS full_name, salary, department_id
FROM employees
WHERE salary > ALL (SELECT salary FROM employees WHERE department_id = 5);
```

Q7. Employees whose salary is greater than at least one employee in department 5. Use ANY/SOME.

```
SELECT employee_id, CONCAT(first_name, ' ', last_name) AS full_name, salary, department_id
FROM employees
WHERE salary > ANY (SELECT salary FROM employees WHERE department_id = 5);
```

Q8. Employees whose salary does not match any salary in department 5.

```
SELECT employee_id, CONCAT(first_name, ' ', last_name) AS full_name, salary, department_id
FROM employees
WHERE salary <> ALL (SELECT salary FROM employees WHERE department_id = 5);
```

Q9. Employees earning above their own department's average. Correlated subquery in WHERE, no join. Display last_name, salary, department_id.

```
SELECT e.last_name, e.salary, e.department_id
FROM employees e
WHERE e.salary > (
    SELECT AVG(salary) FROM employees e2
    WHERE e2.department_id = e.department_id
);
```

Q10. Employees below their department average but above the overall company average.

```
SELECT e.employee_id, CONCAT(e.first_name, ' ', e.last_name) AS full_name,
       e.salary, d.department_name
FROM employees e JOIN departments d ON e.department_id = d.department_id
WHERE e.salary < (SELECT AVG(salary) FROM employees e2 WHERE e2.department_id = e.department_id)
AND e.salary > (SELECT AVG(salary) FROM employees);
```

Section B — Department-Wise (11–20)

Q11. Highest-paid employee(s) per department. Sort by department_name asc.

```
SELECT e.employee_id, CONCAT(e.first_name, ' ', e.last_name) AS full_name,
       e.salary, d.department_name
FROM employees e JOIN departments d ON e.department_id = d.department_id
WHERE e.salary = (SELECT MAX(salary) FROM employees e2 WHERE e2.department_id = e.department_id)
ORDER BY d.department_name ASC;
```

Q12. Lowest-paid employee(s) per department. Include job_title.

```
SELECT e.employee_id, CONCAT(e.first_name, ' ', e.last_name) AS full_name,
       e.salary, d.department_name, j.job_title
FROM employees e
JOIN departments d ON e.department_id = d.department_id
JOIN jobs j ON e.job_id = j.job_id
WHERE e.salary = (SELECT MIN(salary) FROM employees e2 WHERE e2.department_id = e.department_id);
```

Q13. Second-highest salary per department. Exclude departments with fewer than 2 employees.

```
WITH ranked AS (
    SELECT e.employee_id, CONCAT(e.first_name, ' ', e.last_name) AS full_name,
           e.salary, d.department_name, j.job_title,
           DENSE_RANK() OVER (PARTITION BY e.department_id ORDER BY e.salary DESC) AS rnk,
           COUNT(*) OVER (PARTITION BY e.department_id) AS dept_cnt
    FROM employees e
    JOIN departments d ON e.department_id = d.department_id
    JOIN jobs j ON e.job_id = j.job_id
)
SELECT employee_id, full_name, salary, department_name, job_title
FROM ranked WHERE rnk = 2 AND dept_cnt >= 2;
```

Q14. Second-lowest salary per department. Exclude departments with fewer than 2 employees. Sort by department_name asc, salary asc.

```
WITH ranked AS (
    SELECT e.employee_id, CONCAT(e.first_name, ' ', e.last_name) AS full_name,
           e.salary, d.department_name,
           DENSE_RANK() OVER (PARTITION BY e.department_id ORDER BY e.salary ASC) AS rnk,
           COUNT(*) OVER (PARTITION BY e.department_id) AS dept_cnt
    FROM employees e JOIN departments d ON e.department_id = d.department_id
)
SELECT employee_id, full_name, salary, department_name
FROM ranked WHERE rnk = 2 AND dept_cnt >= 2
ORDER BY department_name ASC, salary ASC;
```

Q15. Departments whose average salary exceeds the company overall average. Order by average salary desc.

```
SELECT d.department_id, d.department_name, AVG(e.salary) AS avg_salary
FROM employees e JOIN departments d ON e.department_id = d.department_id
GROUP BY d.department_id, d.department_name
HAVING AVG(e.salary) > (SELECT AVG(salary) FROM employees)
ORDER BY avg_salary DESC;
```

Q16. Departments with max salary > 10,000 and min salary < 5,000.

```
SELECT d.department_id, d.department_name, MIN(e.salary) AS min_salary, MAX(e.salary) AS max_salary
FROM employees e JOIN departments d ON e.department_id = d.department_id
GROUP BY d.department_id, d.department_name
HAVING MAX(e.salary) > 10000 AND MIN(e.salary) < 5000;
```

Q17. Departments with more than 5 employees and average salary > 6,000.

```
SELECT d.department_id, d.department_name, COUNT(*) AS emp_count, AVG(e.salary) AS avg_salary
FROM employees e JOIN departments d ON e.department_id = d.department_id
GROUP BY d.department_id, d.department_name
HAVING COUNT(*) > 5 AND AVG(e.salary) > 6000;
```

Q18. Department(s) with the highest average salary.

```
SELECT department_name, avg_salary FROM (
    SELECT d.department_name, AVG(e.salary) AS avg_salary
    FROM employees e JOIN departments d ON e.department_id = d.department_id
    GROUP BY d.department_id, d.department_name
) t
WHERE avg_salary = (
    SELECT MAX(avg_salary) FROM (
        SELECT AVG(salary) AS avg_salary FROM employees GROUP BY department_id
    ) x
);
```

Q19. Department(s) with the second-highest average salary. No LIMIT.

```
WITH dept_avg AS (
    SELECT d.department_name, AVG(e.salary) AS avg_salary,
           DENSE_RANK() OVER (ORDER BY AVG(e.salary) DESC) AS rnk
    FROM employees e JOIN departments d ON e.department_id = d.department_id
    GROUP BY d.department_id, d.department_name
)
SELECT department_name, avg_salary FROM dept_avg WHERE rnk = 2;
```

Q20. Departments whose employee count exceeds the average employee count across departments.

```
SELECT department_id, department_name, emp_count FROM (
    SELECT d.department_id, d.department_name, COUNT(e.employee_id) AS emp_count
    FROM departments d JOIN employees e ON e.department_id = d.department_id
    GROUP BY d.department_id, d.department_name
) t
WHERE emp_count > (
    SELECT AVG(cnt) FROM (SELECT COUNT(*) AS cnt FROM employees GROUP BY department_id) y
);
```

Section C — EXISTS / Correlated Subquery (21–28)

Q21. Employees in departments whose average salary > 7,000. Use EXISTS.

```
SELECT e.employee_id, CONCAT(e.first_name, ' ', e.last_name) AS full_name,
       e.salary, d.department_name
FROM employees e JOIN departments d ON e.department_id = d.department_id
WHERE EXISTS (
    SELECT 1 FROM employees e2 WHERE e2.department_id = e.department_id
    GROUP BY e2.department_id HAVING AVG(e2.salary) > 7000
);
```

Q22. Employees in departments having at least one employee earning > 12,000. Correlated EXISTS.

```
SELECT e.employee_id, CONCAT(e.first_name, ' ', e.last_name) AS full_name,
       e.salary, d.department_name
FROM employees e JOIN departments d ON e.department_id = d.department_id
WHERE EXISTS (
    SELECT 1 FROM employees e2
    WHERE e2.department_id = e.department_id AND e2.salary > 12000
);
```

Q23. Employees who are the only one in their department earning > 10,000.

```
SELECT e.employee_id, CONCAT(e.first_name, ' ', e.last_name) AS full_name,
       e.salary, d.department_name
FROM employees e JOIN departments d ON e.department_id = d.department_id
WHERE e.salary > 10000
      AND NOT EXISTS (
        SELECT 1 FROM employees e2
        WHERE e2.department_id = e.department_id AND e2.salary > 10000
        AND e2.employee_id <> e.employee_id
      );
```

Q24. Departments with at least one employee above company average salary. Use EXISTS.

```
SELECT d.department_id, d.department_name
FROM departments d
WHERE EXISTS (
  SELECT 1 FROM employees e
  WHERE e.department_id = d.department_id
        AND e.salary > (SELECT AVG(salary) FROM employees)
);
```

Q25. Departments where every employee earns more than 4,000.

```
SELECT d.department_id, d.department_name
FROM departments d
WHERE NOT EXISTS (
  SELECT 1 FROM employees e
  WHERE e.department_id = d.department_id AND e.salary <= 4000
);
```

Q26. Employees in departments that have no employee earning below 3,000.

```
SELECT e.employee_id, CONCAT(e.first_name, ' ', e.last_name) AS full_name,
       e.salary, d.department_name
FROM employees e JOIN departments d ON e.department_id = d.department_id
WHERE NOT EXISTS (
  SELECT 1 FROM employees e2
  WHERE e2.department_id = e.department_id AND e2.salary < 3000
);
```

Q27. Departments with no employee earning more than 15,000. Use NOT EXISTS.

```
SELECT d.department_id, d.department_name
FROM departments d
WHERE NOT EXISTS (
  SELECT 1 FROM employees e
  WHERE e.department_id = d.department_id AND e.salary > 15000
);
```

Q28. Employees with a salary-history record higher than their current salary. Correlated subquery against salaries table.

```
SELECT e.employee_id, CONCAT(e.first_name, ' ', e.last_name) AS full_name, e.salary
FROM employees e
WHERE EXISTS (
  SELECT 1 FROM salaries s
  WHERE s.employee_id = e.employee_id AND s.salary > e.salary
);
```

Section D — INTERSECT / UNION / EXCEPT (29–35)

Q29. Departments with >3 employees AND avg salary > 6,000 (INTERSECT). Display employees in those departments.

```
WITH qualifying_depts AS (
  SELECT department_id FROM employees GROUP BY department_id HAVING COUNT(*) > 3
  INTERSECT
  SELECT department_id FROM employees GROUP BY department_id HAVING AVG(salary) > 6000
)
SELECT e.employee_id, CONCAT(e.first_name, ' ', e.last_name) AS full_name,
       d.department_name, e.salary
FROM employees e JOIN departments d ON e.department_id = d.department_id
WHERE e.department_id IN (SELECT department_id FROM qualifying_depts);
```

Q30. Departments with >=5 employees AND max salary > 12,000 (INTERSECT).

```
WITH qualifying_depts AS (  
    SELECT department_id FROM employees GROUP BY department_id HAVING COUNT(*) >= 5  
    INTERSECT  
    SELECT department_id FROM employees GROUP BY department_id HAVING MAX(salary) > 12000  
)  
SELECT e.employee_id, CONCAT(e.first_name, ' ', e.last_name) AS full_name,  
       e.salary, d.department_name  
FROM employees e JOIN departments d ON e.department_id = d.department_id  
WHERE e.department_id IN (SELECT department_id FROM qualifying_depts);
```

Q31. Departments with >10 employees OR avg salary > 8,000. Use UNION.

```
SELECT department_id, department_name FROM departments  
WHERE department_id IN (SELECT department_id FROM employees GROUP BY department_id HAVING COUNT(*) > 10)  
UNION  
SELECT department_id, department_name FROM departments  
WHERE department_id IN (SELECT department_id FROM employees GROUP BY department_id HAVING AVG(salary) > 8000);
```

Q32. Employees in department 5 OR earning more than 10,000. Use UNION.

```
SELECT employee_id, CONCAT(first_name, ' ', last_name) AS full_name, salary  
FROM employees WHERE department_id = 5  
UNION  
SELECT employee_id, CONCAT(first_name, ' ', last_name) AS full_name, salary  
FROM employees WHERE salary > 10000;
```

Q33. Departments that have employees but no manager assigned. Use EXCEPT.

```
SELECT DISTINCT d.department_id, d.department_name  
FROM departments d JOIN employees e ON e.department_id = d.department_id  
EXCEPT  
SELECT department_id, department_name FROM departments WHERE manager_id IS NOT NULL;
```

Q34. Employees who belong to a department but are not department managers. Use EXCEPT.

```
SELECT employee_id, CONCAT(first_name, ' ', last_name) AS full_name FROM employees  
EXCEPT  
SELECT e.employee_id, CONCAT(e.first_name, ' ', e.last_name)  
FROM employees e JOIN departments d ON e.employee_id = d.manager_id;
```

Q35. Departments with average salary > 6,500 AND max salary > 13,000. Use INTERSECT.

```
SELECT d.department_id, d.department_name, AVG(e.salary) AS avg_salary, MAX(e.salary) AS max_salary  
FROM employees e JOIN departments d ON e.department_id = d.department_id  
GROUP BY d.department_id, d.department_name  
HAVING AVG(e.salary) > 6500  
  
INTERSECT  
  
SELECT d.department_id, d.department_name, AVG(e.salary) AS avg_salary, MAX(e.salary) AS max_salary  
FROM employees e JOIN departments d ON e.department_id = d.department_id  
GROUP BY d.department_id, d.department_name  
HAVING MAX(e.salary) > 13000;
```

Section E — CASE / Conditional Logic (36–40)

Q36. Classify every employee's salary tier: High (>10,000), Medium (6,000–10,000), Low otherwise. Order by salary desc.

```
SELECT employee_id, CONCAT(first_name, ' ', last_name) AS full_name, salary,  
       CASE  
           WHEN salary > 10000 THEN 'High'  
           WHEN salary BETWEEN 6000 AND 10000 THEN 'Medium'  
           ELSE 'Low'  
       END AS salary_tier  
FROM employees  
ORDER BY salary DESC;
```

Q37. Classify each department: Large (>10 employees), Medium (5–10), Small otherwise.

```
SELECT d.department_name, COUNT(e.employee_id) AS employee_count, AVG(e.salary) AS avg_salary,
       CASE
         WHEN COUNT(e.employee_id) > 10 THEN 'Large'
         WHEN COUNT(e.employee_id) BETWEEN 5 AND 10 THEN 'Medium'
         ELSE 'Small'
       END AS department_category
FROM departments d JOIN employees e ON e.department_id = d.department_id
GROUP BY d.department_id, d.department_name;
```

Q38. Salary status per employee: Top Earner (>12,000), Above Average (> company avg), Below Average otherwise. CASE must contain a subquery.

```
SELECT CONCAT(first_name, ' ', last_name) AS full_name, salary,
       CASE
         WHEN salary > 12000 THEN 'Top Earner'
         WHEN salary > (SELECT AVG(salary) FROM employees) THEN 'Above Average'
         ELSE 'Below Average'
       END AS salary_status
FROM employees;
```

Q39. Classify departments: Premium (avg > 8,000), Standard (5,000–8,000), Budget otherwise. Order by avg salary desc.

```
SELECT d.department_name, AVG(e.salary) AS avg_salary,
       CASE
         WHEN AVG(e.salary) > 8000 THEN 'Premium'
         WHEN AVG(e.salary) BETWEEN 5000 AND 8000 THEN 'Standard'
         ELSE 'Budget'
       END AS category
FROM departments d JOIN employees e ON e.department_id = d.department_id
GROUP BY d.department_id, d.department_name
ORDER BY avg_salary DESC;
```

Q40. Classify each employee's salary versus their own department average: Above / Equal / Below. CASE + correlated subquery.

```
SELECT e.employee_id, CONCAT(e.first_name, ' ', e.last_name) AS full_name, e.salary,
       CASE
         WHEN e.salary > (SELECT AVG(salary) FROM employees e2 WHERE e2.department_id = e.department_id)
          THEN 'Above Department Average'
         WHEN e.salary = (SELECT AVG(salary) FROM employees e2 WHERE e2.department_id = e.department_id)
          THEN 'Equal to Department Average'
         ELSE 'Below Department Average'
       END AS salary_comparison
FROM employees e;
```

Section F — Window Functions (41–46)

Q41. Rank all employees by salary, highest to lowest. Use RANK().

```
SELECT employee_id, CONCAT(first_name, ' ', last_name) AS full_name, salary,
       RANK() OVER (ORDER BY salary DESC) AS salary_rank
FROM employees;
```

Q42. Rank employees within each department by salary desc. Use PARTITION BY.

```
SELECT e.employee_id, CONCAT(e.first_name, ' ', e.last_name) AS full_name,
       d.department_name, e.salary,
       RANK() OVER (PARTITION BY e.department_id ORDER BY e.salary DESC) AS dept_salary_rank
FROM employees e JOIN departments d ON e.department_id = d.department_id;
```

Q43. Highest-paid employee(s) per department using a window function.

```
WITH ranked AS (
  SELECT e.employee_id, CONCAT(e.first_name, ' ', e.last_name) AS full_name,
         d.department_name, e.salary,
         RANK() OVER (PARTITION BY e.department_id ORDER BY e.salary DESC) AS rnk
  FROM employees e JOIN departments d ON e.department_id = d.department_id
)
SELECT employee_id, full_name, department_name, salary FROM ranked WHERE rnk = 1;
```

Q44. Second-highest-paid employee(s) per department using a window function.

```
WITH ranked AS (
    SELECT e.employee_id, CONCAT(e.first_name, ' ', e.last_name) AS full_name,
           d.department_name, e.salary,
           DENSE_RANK() OVER (PARTITION BY e.department_id ORDER BY e.salary DESC) AS salary_rank
    FROM employees e JOIN departments d ON e.department_id = d.department_id
)
SELECT employee_id, full_name, department_name, salary, salary_rank
FROM ranked WHERE salary_rank = 2;
```

Q45. Assign a sequential number to employees within each department by salary desc. Use ROW_NUMBER().

```
SELECT employee_id, department_name, salary, row_num FROM (
    SELECT e.employee_id, d.department_name, e.salary,
           ROW_NUMBER() OVER (PARTITION BY e.department_id ORDER BY e.salary DESC) AS row_num
    FROM employees e JOIN departments d ON e.department_id = d.department_id
) t;
```

Q46. For every employee show department average salary and their difference from it. Use a window function.

```
SELECT e.employee_id, CONCAT(e.first_name, ' ', e.last_name) AS full_name,
       d.department_name, e.salary,
       AVG(e.salary) OVER (PARTITION BY e.department_id) AS dept_avg_salary,
       e.salary - AVG(e.salary) OVER (PARTITION BY e.department_id) AS diff_from_avg
FROM employees e JOIN departments d ON e.department_id = d.department_id;
```

Section G — Advanced Combinations (47–50)

Q47. Employees earning above their department average, in departments larger than the average department size.

```
WITH dept_stats AS (
    SELECT department_id, COUNT(*) AS dept_count, AVG(salary) AS dept_avg
    FROM employees GROUP BY department_id
),
overall_avg_size AS (
    SELECT AVG(dept_count) AS avg_size FROM dept_stats
)
SELECT e.employee_id, CONCAT(e.first_name, ' ', e.last_name) AS full_name,
       d.department_name, e.salary, ds.dept_count
FROM employees e
JOIN departments d ON e.department_id = d.department_id
JOIN dept_stats ds ON ds.department_id = e.department_id
CROSS JOIN overall_avg_size oas
WHERE e.salary > ds.dept_avg AND ds.dept_count > oas.avg_size;
```

Q48. Per department, the highest-paid employee among those earning above the company average. Exclude departments with none.

```
WITH above_avg AS (
    SELECT e.*, RANK() OVER (PARTITION BY e.department_id ORDER BY e.salary DESC) AS rnk
    FROM employees e
    WHERE e.salary > (SELECT AVG(salary) FROM employees)
)
SELECT a.employee_id, CONCAT(a.first_name, ' ', a.last_name) AS full_name,
       d.department_name, a.salary
FROM above_avg a JOIN departments d ON a.department_id = d.department_id
WHERE a.rnk = 1;
```

Q49. Employees in departments with >3 employees AND avg salary > company avg (INTERSECT). Tag Top Tier (>10,000) / Standard Tier. Order by department_name asc, salary desc.

```
WITH qualifying_depts AS (
    SELECT department_id FROM employees GROUP BY department_id HAVING COUNT(*) > 3
    INTERSECT
    SELECT department_id FROM employees GROUP BY department_id
    HAVING AVG(salary) > (SELECT AVG(salary) FROM employees)
)
SELECT e.employee_id, CONCAT(e.first_name, ' ', e.last_name) AS full_name,
       d.department_name, e.salary,
       CASE WHEN e.salary > 10000 THEN 'Top Tier' ELSE 'Standard Tier' END AS salary_tier
FROM employees e JOIN departments d ON e.department_id = d.department_id
WHERE e.department_id IN (SELECT department_id FROM qualifying_depts)
ORDER BY d.department_name ASC, e.salary DESC;
```

Q50. FULL MOCK: departments with >3 employees, avg salary > company avg, at least one employee > 10,000, and an assigned manager. Top 2 earners per department, ranked and tiered. Must use a CTE, a window function, and a CASE expression; no LIMIT/TOP/FETCH.

```
WITH dept_stats AS (
    SELECT department_id, COUNT(*) AS emp_count, AVG(salary) AS avg_salary, MAX(salary) AS max_salary
    FROM employees GROUP BY department_id
),
qualifying_depts AS (
    SELECT ds.department_id
    FROM dept_stats ds JOIN departments d ON d.department_id = ds.department_id
    WHERE ds.emp_count > 3
        AND ds.avg_salary > (SELECT AVG(salary) FROM employees)
        AND ds.max_salary > 10000
        AND d.manager_id IS NOT NULL
),
ranked AS (
    SELECT e.employee_id, CONCAT(e.first_name, ' ', e.last_name) AS full_name,
        d.department_name, j.job_title, e.salary,
        RANK() OVER (PARTITION BY e.department_id ORDER BY e.salary DESC) AS salary_rank
    FROM employees e
    JOIN departments d ON e.department_id = d.department_id
    JOIN jobs j ON e.job_id = j.job_id
    WHERE e.department_id IN (SELECT department_id FROM qualifying_depts)
)
SELECT employee_id, full_name, department_name, job_title, salary, salary_rank,
    CASE WHEN salary > 10000 THEN 'Top Tier' ELSE 'Standard Tier' END AS salary_tier
FROM ranked
WHERE salary_rank <= 2
ORDER BY department_name ASC, salary DESC;
```